KNU 강원대학교
KANGWON NATIONAL UNIVERSITY

R E S U L T R E P O R T

마이크로프로세서응용및실험 결과보고서_타이머카운터



수 강 과 목	: 마이크로프로세서응용및실험-2분반
담 당 교 수	: 김도인 교수님
학 번	: 202210820
작 성 자	: 이동제
공 동 실 험 자	: 강민수(202214221)
제 출 일	: 2024-05-23 목요일

<목 차>

1. 실험목적 ····· 2

2. 실험원리 ····· 2

3. 실험기구 및 재료 ····· 3

4. 실험방법 ····· 3-6

5. 측정값 ····· 6

6. 실험결과 ····· 7

7. 결과에 대한 논의 ····· 8

8. 결론 ····· 8

9. 참고문헌 및 출처 ····· 8

1. 실험목적

Timer/Counter 사용 방법을 익힌다.

2. 실험원리

타이머/카운터(Timer/Counter)란 어떠한 클록 펄스를 카운트(Count)하는 장치를 말합니다.
이것은 타이머와 카운터로 각각 동작시킬 수 있지만, 타이머/카운터 동작을 동시에 시킬 수는 없습니다.

타이머로 동작 시, 마이크로컨트롤러 내부의 시스템 클럭을 카운트합니다.
카운터로 동작 시, 마이크로컨트롤러 외부의 입력 클럭 또는 이벤트를 카운트합니다.

제작한 ATmega128에는 8비트 타이머/카운터와 16비트 타이머/카운트가 존재하지만, 이번 실습에서는 8비트만 사용합니다.

8비트 타이머 특징
- 클록이 입력될 때마다 값이 1씩 증가/감소함. - 카운터 입력 클록은 TOSC1 단자로부터 입력된 것을 사용할 수 있음. - 오버플로우 인터럽트, 비교매치 인터럽트를 선택해서 사용할 수 있음.

표 1) 8비트 타이머의 특징

오버플로우 인터럽트(Overflow Interrupt): TCNT0의 값이 최댓값 255이 되어 오버 플로우가 발생하면 인터럽트 발생
비교매치 인터럽트(Compare Match Event Interrupt): TCNT0의 값과 OCR0의 값이 일치할 경우 인터럽트 발생.

8비트 타이머/카운트는 Timer/Count0과 Timer/Count2로 나뉘어집니다.
Timer/Count0을 제어하기 위한 레지스터는 TCNT0, TCCR0, OCR0, TIMSK, TIFR의 레지스터의 비트를 바꿔 제어합니다.

TCNT0 – 카운터로 사용되는 레지스터로, 직접 쓰거나 읽을 수 있음.
TCCR0 – Time clock clk_{T0}의 정밀도,. 타이머의 동작 모드, 비교 매치 동작일 때 OC0 핀의 동작을 선택할 수 있음.
OCR0 – 항상 카운터 TCNT0의 값과 비교되는 8비트 데이터가 저장되어 있음.
TIMSK – 인터럽트 발생을 오버플로우 인터럽트와 비교매치 인터럽트 중 선택할 수 있음.
TIFR – 오버플로우와 비교매치 인터럽트를 발생을 알려줌.

이러한 실험 원리들을 가지고 타이머/카운터 기능을 통해 변화되는 레지스터 값을 가지고 LCD 화면에 문자열을 출력하는 테스트를 진행합니다.

3. 실험기구 및 재료

실험기구 및 재료	수량
ATmega128 마이크로 컨트롤러	1개
컴퓨터	1개
ISP 케이블	1개

표 2) 실험기구 및 재료

4. 실험방법

타이머/카운터 기능을 사용하는 테스트를 진행하기 위해 AVR Studio에서 코드를 작성합니다.

이번 실습교재8의 예제3번(파형출력)을 제외한 예제 1,2,4번의 코드를 통해 Timer/Counter0을 사용하여 Overflow Interrupt과 비교매치 인터럽트를 통해 LED를 1초 간격으로 점멸시키고, 외부 클록 T2와 Timer/Counter2를 사용하여 LCD에 발생 횟수를 띄워 스위치를 누를 때마다 증가하는 것을 관찰했습니다.

후에 구술할 실험방법은 추가로 진행된 실습에 대해 작성하겠습니다.

1. 8-bit Timer/Counter0와 Timer/Counter2의 차이점을 알아오시오.

타이머/카운터0과 타이머/카운터2의 공통점은 8비트 타이머/카운터이며, 4가지 동작 모드를 지원합니다.

하지만 차이점은 타이머/카운터0은 프리스케일러가 1, 8, 32, 64, 128, 256, 1024로 다양한 편이지만 타이머/카운터2는 1, 8, 64, 256, 1024로 비교적 가짓수가 적습니다.

타이머/카운터2는 0과 다르게 External Event Counter라는 것이 별도로 존재합니다. External Event Counter는 외부에서 들어오는 이벤트를 카운트하는 것으로, 스위치를 한 번 동작하면 레지스터가 증가하는 동작을 말합니다.

그래서 Timer/Counter0은 카운트 입력핀이 외부 클록에 의해 레지스터가 증가하는 TOSC1 핀을 사용하고, Timer/Counter2의 카운트 입력핀은 외부 이벤트에 의해 레지스터가 증가하는 T2 핀을 사용합니다.

2. 타이머/카운터와 LCD를 이용한 Digital 시계를 프로그램 하시오.
※ Tip ➔ 시, 분, 초를 계산할 때 ' / '와 ' % '의 연산자를 사용하시오.

```
#include<avr/io.h>
#include<avr/interrupt.h>
#include<util/delay.h>
```

입출력 포트 및 레지스터에 대한 정의를 시키고, 지연(Delay) 함수와 인터럽트(interrupt) 함수를 사용하기 위해서 AVR 라이브러리 속 헤더 파일을 포함하는 코드를 입력합니다.

```
void lcd_inst(unsigned char value) // 명령어 입력
{
        PORTA &= ~0x02;
        _delay_ms(1);
        PORTA |= 0x08;
        _delay_ms(1);
        PORTC=value;
        _delay_ms(1);
        PORTA &= 0x00;
        _delay_ms(1);
}

void lcd_init() // LCD 초기화
{
        _delay_ms(50);
        lcd_inst(0x38);
        _delay_ms(1);
        lcd_inst(0b00001100);
        _delay_ms(1);
        lcd_inst(0x01);
        _delay_ms(1);
        lcd_inst(0b00000110);
        _delay_ms(1);
}

void lcd_ch(unsigned char ch)
{
        PORTA |= 0x02;
        _delay_ms(1);
        PORTA |= 0x08;
        _delay_ms(1);
        PORTC=ch;
        _delay_ms(1);
        PORTA &= 0x00;
        _delay_ms(1);
}

void lcdout(unsigned char add, unsigned char cc)
{
        lcd_inst(add | 0x80);
        _delay_ms(1);
        lcd_ch(cc);
}

void lcd_print(char line, char *text)
{
        lcd_inst(line | 0x80);
        while(*text)

        lcd_ch(*text++);
}
```

그리고 타이머/카운트 기능의 동작을 LCD 화면에 출력하기 위해
존재하지 않는 LCD 헤더 파일 대신 LCD 함수를 입력합니다.

```
int Cnt=0;
```

```
int t = 0;
int k=0;
int sec=54;
int min=59;
int hour=23;

char temp[8] = {'0', '0', ':', '0', '0', ':', '0', '0'};
```

Cnt와 t, k, sec, min, hour라는 변수에 값을 대입하고 그 변수를 사용한다고 선언합니다.
sec는 '초', min은 '분', hour는 '시간'을 나타내기 위한 변수입니다.
그리고 temp를 8자리로 표현하다고 하고, 구분을 위해 시간과 분, 분과 초 사이를 콜론으로 적어둡니다.

```
ISR(TIMER0_OVF_vect)            // overflow interrupt service routine.
{
        t++;

        if(t==125)
        {
        sec++;
        t=0;
        }
        if(sec==60)
        {
        min++;
        sec=0;
        }
        if(min==60)
        {
        hour++;
        min=0;
        }
        if(hour==24)
        {
        hour=0;
        }


TCNT0=130;


}
```

타이머0이 overflow되었을 때 발생하는 인터럽트 함수에 각 IF문을 만족할 시 sec를 더하고, min을 더하고, hour을 더하다가 초, 분, 시간이 일정 값을 넘어서면 돌아올 수 있도록 만들어 줍니다.
타이머가 125번 오버플로우되면 1초가 경과한 것으로 간주합니다.
min++; 함수는 min=min+1;과 같은 의미입니다.

```
int main(void)
{
        DDRB=0xff;
        PORTB=0xff;
        DDRC=0xff;
        DDRA=0x0a;

        TCCR0=0x07;       // 외부 클록으로 설정
        TCNT0=130;     // 카운터를 130으로 초기화
```

```
        TIMSK=0x01;         // timer/counter0 overflolw 인터럽트 발생을 허용함.
        SREG |=0x80;

        lcd_init();
        _delay_ms(20);

        while(1)
        { lcd_print(0x00, "TIME -");

        temp[0]= hour / 10 + 48;
        temp[1]= hour % 10 + 48;
        temp[3]= min / 10 + 48;
        temp[4]= min % 10 + 48;
        temp[6]= sec / 10 + 48;
        temp[7]= sec % 10 + 48;

                for (k=0;k<8;k++)
                {
                lcdout(0x07+k, temp[k]);
                }
        }
}
```

메인 함수로 TCCR0=0x07;로 두어 외부 클록으로 레지스터를 증가시키도록 하고, TISMK 레지스터는 타이머0 Overflow 인터럽트를 사용하며 인터럽트를 허용하겠다고 선언합니다.
While문을 통해 lce_print(0x00, “Time -”);을 통해 “TIME -”라는 문자열이 계속해서 출력되고, temp 0, 1, 3, 4, 6, 7을 / 연사자와 % 연산자를 통해 위에서 외부 클럭에 의해 증가하는 sec, min hour 값을 몫과 나머지로 나눠 십의 자리, 일의 자리에 각각 대입될 수 있도록 합니다.
그리고 for문으로 temp 0~7까지 lcdout 함수를 통해 LCD에 출력되도록 합니다.

5. 측정값

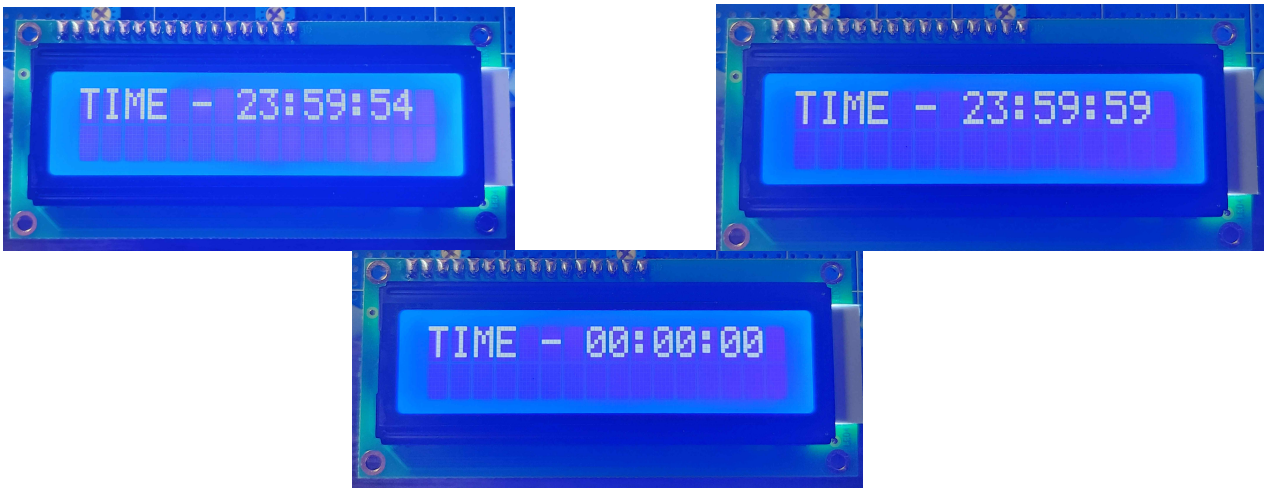
초기 상태를 지정하고 외부 클럭에 반응하고, 오버플로우 인터럽트를 통해 변화하는 LCD 문자열을 출력하도록 만들었다.

타이머/카운터를 사용한 LCD 제어

초기 상태	LCD 화면에 ‘TIME - 23 : 59 : 54’라는 문자열이 출력됨
시간이 흐름에 따라	sec에 속해있는 54가 1씩 증가하기 시작함. 59를 넘어 60이 되자 0으로 변하며 앞에 min부분을 1씩 증가시킴. (=min도 sec과 똑같이 동작함.)
시간이 흐른 후	hour도 min이 60으로 넘어가자 23에서 24로 변하지 않고 바로 00으로 변함.

표 3) 타이머/카운터 기능을 사용하는 코딩 테스트 측정값

6. 실험결과



작성한 코드가 타이머/카운트 기능을 사용해 LCD 화면에 클럭에 따라 변동하며 원하는 문자열을 띄우는지 테스트하였다.

첫 번째 테스트, “TIME -“을 출력하고 temp를 통해 8자리 문자열을 지정을 해주었다. 그리고 클럭에 따라 증가하는 값을 통해 sec값을 증가시킬 수 있었다. 하지만 sec값이 10의 자리를 넘어가지 않고 1의 자리에서 ASCII Table에 있는 문자로 계속해서 변화하는 오류가 발생했다.

두 번째 테스트, 첫 번째 문제점을 해결하기 위해 증가하는 sec값을 % 연산자와 / 연산자를 통해 나누어서 10의 자리에는 몫을, 1의 자리에는 나머지를 입력하도록 만들어 해결하였다. 그리고 sec가 60을 넘어서도 꾸준히 증가하는 문제점이 발생했다.

마지막 테스트, 두 번째 문제점을 해결하기 위해 IF문을 통해 sec가 60이 되면 0으로 변하고, min을 1씩 증가시키도록 하여 해결하였다. 같은 방법을 hour에도 적용하고 hour는 24가 되면 0이 되도록 설정했다.

정상적으로 카운터 동작을 수행하였다.

7. 결과에 대한 논의

아쉬웠던 점은 저번 실습 때 사용하였던 IF문을 통해 일정값을 넘으면 동작하지 않는 점을 이용하여 sec, min, hour가 적절히 동작하도록 배치하는 과정을 뒤늦게 깨달았다는 것이다.

좋았던 점은 저번 실습의 경험을 살려 "TIME -" 문자열을 비교적 쉽게 띄울 수 있었다는 점이다. 그리고 % 연산자와 / 연산자를 통해 증가하는 레지스터 값을 나누어서 각각 출력할 수 있다는 점이 놀라웠다.

만약 디지털 시계를 제작한다면 이러한 원리로 작동하겠다는 것을 깨달아서 더욱 친근하게 다가왔다.
sec가 증가하면 min이, min이 증가하면 hour가 증가하는 이러한 흥미로운 연쇄 작용은 이번 실습 코드를 만드는 과정에 있어서 더욱 흥미롭게 집중되었던 이유 중 하나였다.

8. 결론

ATmega128 마이크로 컨트롤러 보드의 Timer/Counter 기능을 제어하는 레지스터와 레지스터에 원하는 비트를 입력해 동작 모드를 제어하고, 외부 클럭이나 이벤트로 인해 변화하는 레지스터의 값을 사용하여 LCD 화면에 출력하는 방법을 배웠다. 따라서 직접 스위치를 눌러도 되지만, 누르지 않고도 보드의 자체 기능을 통해 동작할 수 있음을 깨닫게 되었다.

9. 참고문헌 및 출처

(교재) 김도인 교수님,「실습교재8」, 강원대학교, 전기전자공학과, (2~15p)
(교재) 김도인 교수님 [2024] 마프_Part 08_타이머/카운터_ver2, " ", (4~36p)
(사진) 이동제와 강민수, 실습 중 찍은 사진, 2024-05-16